

Module : Apprentissage Artificiel (Machine Learning)

Filière Parcours d'Excellence ISOC

Semestre 6

TP2 : La Régression Linéaire Simple

Responsable : Brahim AKSASSE
FS Meknès
Département Informatique

Année Universitaire : 2025-2026

1. Introduction

Le but de ce TP de programmation est de vous familiariser avec l'apprentissage automatique. Les exercices sont conçus pour vous donner une expérience pratique pour faire fonctionner ces algorithmes. Nous allons les programmer avec Matlab.

Ce premier exercice vous permettra de vous entraîner à la régression linéaire simple. Ces exercices ont été largement testés avec Matlab, mais ils devraient également fonctionner sous autre langage de programmation surtout Python.

2. Données

Téléchargez tp2x.data et tp2y.data par `load tp2x.data` et `load tp2y.data`.

Les fichiers contiennent des exemples de mesures de taille pour divers garçons âgés de deux à huit ans. Les valeurs y sont les hauteurs mesurées en mètres et les valeurs x sont les âges des garçons correspondant aux hauteurs.

Chaque couple de taille et d'âge constitue un exemple d'entraînement $(x^{(i)}, y^{(i)})$, $i = 1, \dots, m$, dans notre ensemble de données. Il y a 50 exemples d'entraînement $m = 50$, et vous les utiliserez pour développer un modèle de régression linéaire.

3. Problème d'apprentissage supervisé

Dans ce problème, vous allez implémenter une régression linéaire à l'aide de l'algorithme **Gradient Descent** (la descente de gradient). Dans Matlab, vous pouvez charger l'ensemble d'entraînement à l'aide des commandes

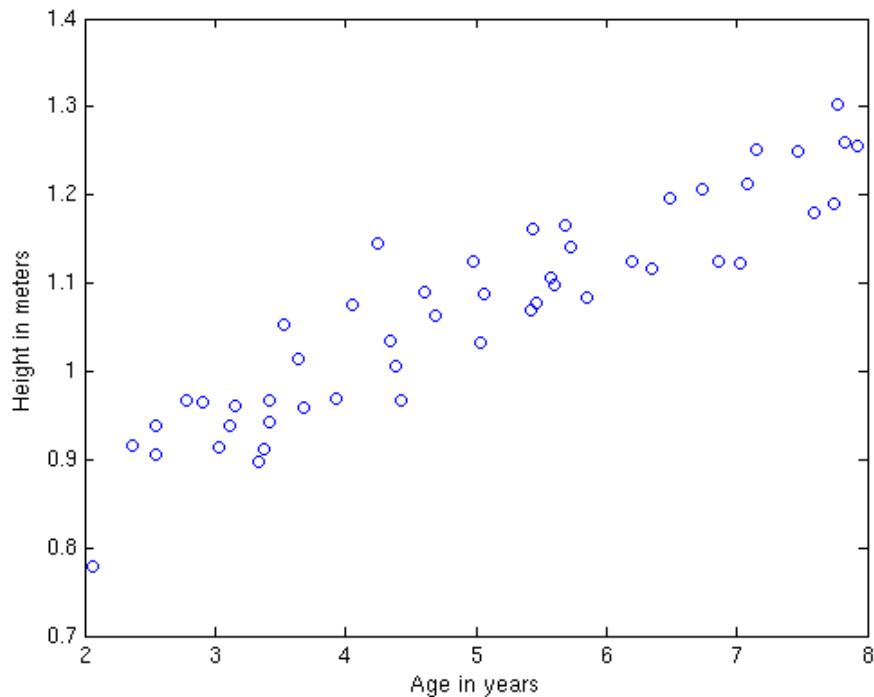
```
x = load ('tp1x.dat');  
y = load ('tp1y.dat');
```

Ce sera notre ensemble d'entraînement pour un problème d'apprentissage supervisé avec des features $n = 1$ ($X_1 = x^{(i)}$ en plus de l'habituel $X_0 = 1$).

Exécutez les commandes suivantes pour tracer votre ensemble d'entraînement (et étiqueter les axes):

```
figure % open a new figure window  
plot(x, y, 'o');  
ylabel('Height in meters')  
xlabel('Age in years')
```

Vous devriez voir une série de points de données similaires à la figure ci-dessous.



Avant de commencer l'apprentissage des paramètres du modèle (fiter le modèle aux données) par l'algorithme **Gradient Descent**, nous devons ajouter le terme d'interception $X_0 = 1$ à chaque exemple. Pour ce faire, on utilise la commande suivante

```
m = length(y); % store the number of training examples
x = [ones(m, 1), x]; % Add a column of ones to x
```

4. La regression linéaire

Maintenant, nous allons implémenter la régression linéaire pour ce problème. Rappelons que le modèle de régression linéaire est

$$\hat{y} = h_{\theta}(x) = \theta^T x = \sum_{j=0}^n \theta_j x_j = \theta_0 + \theta_1 x$$

La règle de mise à jour du vecteur paramètres par Batch Gradient descent est

$$\theta_j = \theta_j - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \quad j = 0, 1, \dots, n$$

- 1) Implémenter l'algorithme du Gradient descent en utilisant différents taux d'apprentissage $\alpha = 0.01, 0.1, 1, 10$.
- 2) Continuez à exécuter l'algorithme du Gradient descent pour plus d'itérations jusqu'à ce que θ converge. (Cela prendra un total d'environ 1500 itérations). Après la convergence, enregistrez les valeurs finales de $\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$ que vous obtenez.

Lorsque vous avez trouvé θ , tracez l'ajustement de la ligne droite de votre algorithme sur le même graphique que vos données d'entraînement. Les commandes de traçage ressembleront à ceci

```
hold on % Plot new data without clearing old plot
plot(x(:,2), x*theta, '-')
% remember that x is now a matrix with 2 columns
```

```
% and the second column contains the time info  
legend('Training data', 'Linear regression')
```

- 3) Enfin, nous aimerions faire quelques prédictions en utilisant l'hypothèse apprise. Utilisez votre modèle pour prédire la taille de deux garçons âgés de 3,5 et 7 ans.

5. La solution par les équations normales

On peut retrouver la solution pour l'apprentissage du vecteur paramètres en utilisant les équations normales.

$$\text{Posons } X = \begin{bmatrix} x^{(1)} & 1 \\ \vdots & \vdots \\ x^{(m)} & 1 \end{bmatrix}, Y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$
$$\hat{\theta} = \frac{1}{m} (X^T X)^{-1} (X^T Y)$$

Comparer les deux estimateurs du vecteur paramètres

6. La solution par la méthode de Newton

La méthode de Newton-Raphson est une méthode itérative mais beaucoup plus rapide que l'algorithme du gradient descent. En quelques itérations, vous allez obtenir la solution pour le vecteur paramètres

$$\theta = \theta - H(\theta)^{-1} \nabla_{\theta} (l(\theta))$$

Comparer les trois estimateurs obtenus pour le vecteur paramètres.